

LAB ASSIGNMENT-11.3

< AI Assisted Coding >

NAME: P.srikar

HALLTICKETNUMBER : 2403A52002

BATCH NUMBER : 01

TASK__1

Task Description #1 – Stack class implementation

Task: Ask AI to implement a stack class with push(), pop(), peek() and is_empty() methods

PROMPT:

Write a Python program to implement a Stack class with the following methods: push(), pop(), peek(), and is_empty(). The program should allow the user to interactively perform stack operations by choosing from a menu.

CODE:

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            return "Stack is empty. Cannot pop."

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            return "Stack is empty. Nothing to peek."

    def is_empty(self):
        return len(self.items) == 0

def main():
    stack = Stack()

    while True:
        print("\n==== Stack Operations Menu =====")
        print("1. Push")
        print("2. Pop")
        print("3. Peek")
        print("4. Is Empty?")
        print("5. Exit")
        choice = input("Enter your choice (1-5): ")
```

```
if choice == '1':
    item = input("Enter item to push: ")
    stack.push(item)
    print(f"'{item}' pushed onto the stack.")

elif choice == '2':
    result = stack.pop()
    print("Popped item:", result)

elif choice == '3':
    result = stack.peek()
    print("Top item:", result)

elif choice == '4':
    print("Is stack empty?", stack.is_empty())

elif choice == '5':
    print("Exiting program.")
    break

else:
    print("Invalid choice. Please enter a number from 1 to 5.")

if __name__ == "__main__":
    main()
```

```
===== Stack Operations Menu =====
1. Push
2. Pop
3. Peek
4. Is Empty?
5. Exit
Enter your choice (1-5): 1
Enter item to push: apple
'apple' pushed onto the stack.

===== Stack Operations Menu =====
1. Push
2. Pop
3. Peek
4. Is Empty?
5. Exit
Enter your choice (1-5): 5
Exiting program.
```

OBSERVATION:

The stack program successfully demonstrates the LIFO (Last In, First Out) principle using basic operations like push, pop, peek, and is_empty. The menu-driven interface allows users to interact with the stack intuitively. Error handling for empty stack conditions ensures smooth execution. Overall, the program is efficient, user-friendly, and meets the expected functionality of a basic stack data structure.

TASK__2

Task Description #2 – Queue Implementation

Task: Use AI to generate a Queue class with enqueue(), dequeue(), and is_empty().

PROMPT:

Create a Python program that implements a Queue class with enqueue(), dequeue(), and is_empty() methods. The program should provide a menu-driven interface to allow the user to perform queue operations interactively.

CODE:

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            return "Queue is empty. Cannot dequeue."

    def is_empty(self):
        return len(self.items) == 0

def main():
    queue = Queue()

    while True:
        print("\n==== Queue Operations Menu =====")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Is Empty?")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            item = input("Enter item to enqueue: ")
            queue.enqueue(item)
            print(f"'{item}' enqueued into the queue.")

        elif choice == '2':
            result = queue.dequeue()
            print("Dequeued item:", result)

        elif choice == '3':
            print("Is queue empty?", queue.is_empty())
```

```
elif choice == '4':
    print("Exiting program.")
    break

else:
    print("Invalid choice. Please enter a number from 1 to 4.")

if __name__ == "__main__":
    main()
```

```
===== Queue Operations Menu =====
1. Enqueue
2. Dequeue
3. Is Empty?
4. Exit
Enter your choice (1-4): 1
Enter item to enqueue: apple
'apple' enqueued into the queue.

===== Queue Operations Menu =====
1. Enqueue
2. Dequeue
3. Is Empty?
4. Exit
Enter your choice (1-4): 2
Dequeued item: apple

===== Queue Operations Menu =====
1. Enqueue
2. Dequeue
3. Is Empty?
4. Exit
Enter your choice (1-4): 3
Is queue empty? True

===== Queue Operations Menu =====
1. Enqueue
2. Dequeue
3. Is Empty?
4. Exit
Enter your choice (1-4): 4
Exiting program.
```

OBSERVATION:

The program implements a queue data structure using Python lists, demonstrating FIFO (First In, First Out) behavior. The enqueue, dequeue, and is_empty methods allow basic queue operations. The menu-driven interface makes it user-friendly. Proper checks prevent errors during dequeuing from an empty queue.

TASK__3

Task Description #3 – Linked List Implementation

Task: Ask AI to create a singly linked list with `insert_at_end()`, `insert_at_beginning()`, and `display()`.

PROMPT:

Develop a Python program that implements a singly linked list with methods `insert_at_end()`, `insert_at_beginning()`, and `display()`. The program should enable the user to insert elements at the beginning or end and display the list through a menu-based interface.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"Inserted '{data}' at the beginning.")

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            print(f"Inserted '{data}' as the first node.")
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
        print(f"Inserted '{data}' at the end.")

    def display(self):
        if self.head is None:
            print("The linked list is empty.")
            return
        current = self.head
        print("Linked List elements:")
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
```

```

def main():
    ll = SinglyLinkedList()

    while True:
        print("\n==== Linked List Menu =====")
        print("1. Insert at Beginning")
        print("2. Insert at End")
        print("3. Display")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            data = input("Enter data to insert at beginning: ")
            ll.insert_at_beginning(data)

        elif choice == '2':
            data = input("Enter data to insert at end: ")
            ll.insert_at_end(data)

        elif choice == '3':
            ll.display()

        elif choice == '4':
            print("Exiting program.")
            break

        else:
            print("Invalid choice. Please enter 1 to 4.")

if __name__ == "__main__":
    main()

```

```

if __name__ == "__main__":
    main()

```



```

==== Linked List Menu =====
1. Insert at Beginning
2. Insert at End
3. Display
4. Exit
Enter your choice (1-4): 1
Enter data to insert at beginning: 245
Inserted '245' at the beginning.

==== Linked List Menu =====
1. Insert at Beginning
2. Insert at End
3. Display
4. Exit
Enter your choice (1-4): 2
Enter data to insert at end: 584
Inserted '584' at the end.

==== Linked List Menu =====
1. Insert at Beginning
2. Insert at End
3. Display
4. Exit
Enter your choice (1-4): 3
Linked List elements:
245 -> 584 -> None

==== Linked List Menu =====
1. Insert at Beginning
2. Insert at End
3. Display
4. Exit
Enter your choice (1-4): 4
Exiting program.

```

OBSERVATION:

This program demonstrates a singly linked list with insertion at both the beginning and end. Nodes are dynamically linked, and the display()

function shows the current list structure. The menu allows for easy user interaction. It effectively shows the use of pointers and node manipulation in linked list data structures.

TASK__4

Task Description #4 – Binary Search Tree (BST)

Task: Ask AI to generate a simple BST with insert() and inorder_traversal().

PROMPT:

Implement a simple Binary Search Tree (BST) in Python with methods insert() and inorder_traversal(). The program should allow the user to insert values and display the tree's in-order traversal using a menu-driven interface.

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        self.root = self._insert_recursive(self.root, data)

    def _insert_recursive(self, current_node, data):
        if current_node is None:
            return Node(data)
        if data < current_node.data:
            current_node.left = self._insert_recursive(current_node.left, data)
        elif data > current_node.data:
            current_node.right = self._insert_recursive(current_node.right, data)
        # If equal, do nothing or handle duplicates as needed
        return current_node

    def inorder_traversal(self):
        print("In-order Traversal (Sorted Order):")
        self._inorder_recursive(self.root)
        print() # for newline

    def _inorder_recursive(self, current_node):
        if current_node is not None:
            self._inorder_recursive(current_node.left)
            print(current_node.data, end=" ")
            self._inorder_recursive(current_node.right)

def main():
    bst = BinarySearchTree()

    while True:
        print("\n==== Binary Search Tree Menu =====")
        print("1. Insert")
        print("2. In-order Traversal")
        print("3. Exit")

def main():
    bst = BinarySearchTree()

    while True:
        print("\n==== Binary Search Tree Menu =====")
        print("1. Insert")
        print("2. In-order Traversal")
        print("3. Exit")

        choice = input("Enter your choice (1-3): ")

        if choice == '1':
            value = int(input("Enter value to insert: "))
            bst.insert(value)
            print(f"Inserted {value} into the BST.")

        elif choice == '2':
            bst.inorder_traversal()

        elif choice == '3':
            print("Exiting program.")
            break

        else:
            print("Invalid choice. Please enter 1, 2, or 3.")

if __name__ == "__main__":
    main()
```

```
===== Binary Search Tree Menu =====
1. Insert
2. In-order Traversal
3. Exit
Enter your choice (1-3): 1
Enter value to insert: 5
Inserted 5 into the BST.

===== Binary Search Tree Menu =====
1. Insert
2. In-order Traversal
3. Exit
Enter your choice (1-3): 1
Enter value to insert: 2
Inserted 2 into the BST.

===== Binary Search Tree Menu =====
1. Insert
2. In-order Traversal
3. Exit
Enter your choice (1-3): 2
In-order Traversal (Sorted Order):
2 5

===== Binary Search Tree Menu =====
1. Insert
2. In-order Traversal
3. Exit
Enter your choice (1-3): 3
Exiting program.
```

OBSERVATION:

The program successfully builds a binary search tree and performs in-order traversal to display elements in sorted order. The recursive insert method ensures values are correctly placed. The in-order traversal validates the BST structure. The menu makes it interactive and user-friendly.